Expt 1 Date: /08/2026

Implement MinMax algorithm.

Aim: To implement the Min-Max algorithm using the Divide and Conquer technique to find the minimum and maximum elements in an array.

Theory:

Algorithms are used to help design programs that perform particular tasks. Algorithms consist of steps that are carried out (performed) one after another. There are three basic building blocks (constructs) to use when designing algorithms: sequencing, selection, and iteration.

The **Min-Max problem** involves finding both the **smallest (minimum)** and the **largest (maximum)** elements from a given array of *n* elements.

A simple approach compares each element with the current minimum and maximum separately. This method requires approximately **2(n − 1)** comparisons in the worst case.

The **Divide and Conquer** technique provides a more efficient solution by dividing the array into smaller subarrays, solving each subproblem recursively, and then combining the results.

The Divide and Conquer strategy consists of three steps:

1. **Divide:** Split the array into two nearly equal halves.
2. **Conquer:** Recursively determine the minimum and maximum values of each half.
3. **Combine:** Compare the minimum values of both halves to obtain the overall minimum and compare the maximum values to obtain the overall maximum.

Base Cases

- **If the array contains only one element**, that element is both the minimum and maximum.
- **If the array contains two elements**, compare them once to determine the minimum and maximum.

Recursive Case

If the array contains more than two elements:

- Divide the array into two halves.
- Recursively find the minimum and maximum of each half.
- Compare the minimum values obtained from both halves to determine the overall minimum.
- Compare the maximum values obtained from both halves to determine the overall maximum.

Algorithm:

```
1   Algorithm MaxMin(i, j, max, min)
2   // a[1 : n] is a global array. Parameters i and j are integers,
3   // 1 ≤ i ≤ j ≤ n. The effect is to set max and min to the
4   // largest and smallest values in a[i : j], respectively.
5   {
6       if (i = j) then max := min := a[i]; // Small(P)
7       else if (i = j − 1) then  // Another case of Small(P)
8           {
9               if (a[i] < a[j]) then
10              {
11                  max := a[j]; min := a[i];
12              }
13              else
14              {
15                  max := a[i]; min := a[j];
16              }
17          }
18          else
19          {   // If P is not small, divide P into subproblems.
20              // Find where to split the set.
21                  mid := ⌊(i + j)/2⌋;
22              // Solve the subproblems.
23                  MaxMin(i, mid, max, min);
24                  MaxMin(mid + 1, j, max1, min1);
25              // Combine the solutions.
26                  if (max < max1) then max := max1;
27                  if (min > min1) then min := min1;
28          }
29  }
```

Algorithm 3.6 Recursively finding the maximum and minimum

Problem:

Using the **Divide and Conquer Min-Max algorithm**, determine the **minimum** and **maximum** elements in the following array.

Index	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
Value	22	13	-5	-8	15	60	17	31	47

Solution:

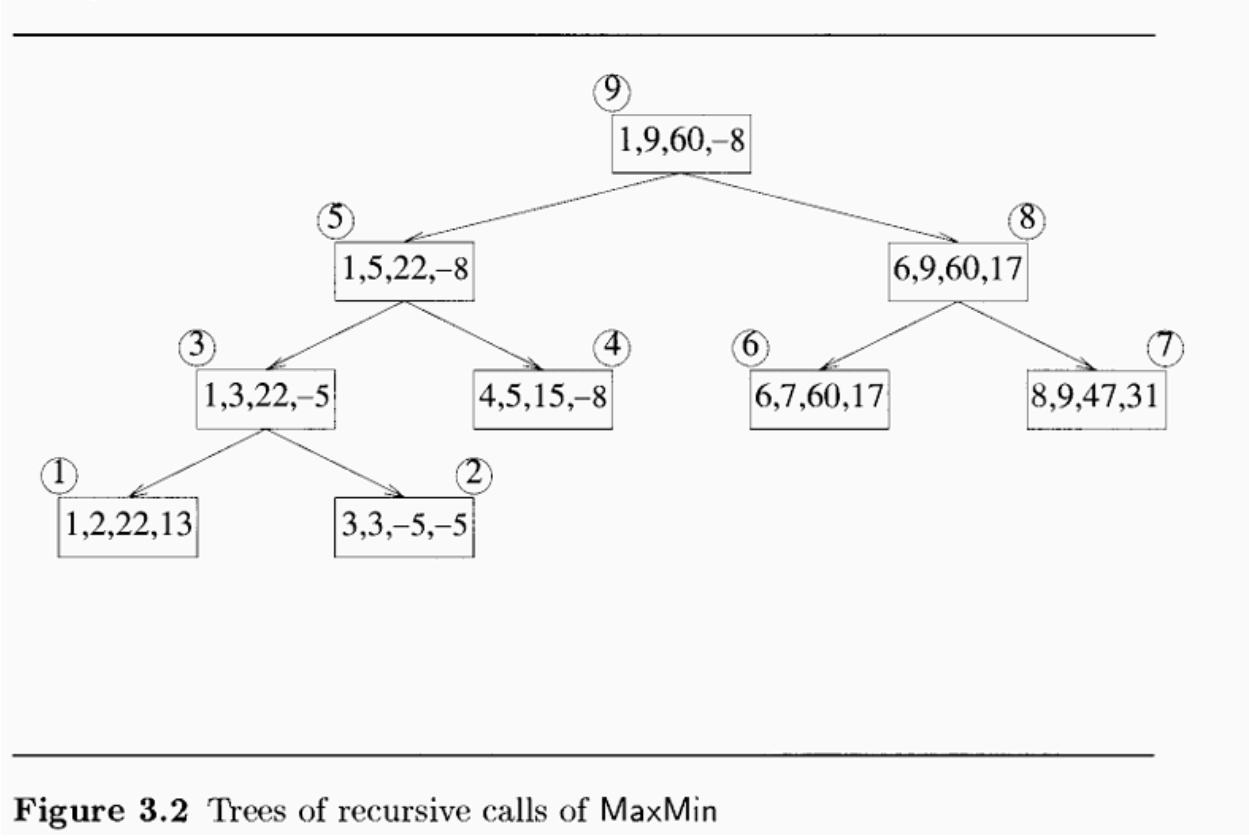


Figure 3.2 Trees of recursive calls of MaxMin

Task : Write a C/C++ program to implement the **MaxMin algorithm using Divide and Conquer** for the array:

A={22,13,−5,−8,15,60,17,31,47}

The program should:

- Find the minimum and maximum elements.
- Display the recursion tree by printing all recursive calls made during execution.**(optional)**.
- Print the final minimum and maximum values.

Code:

Output:

Conclusion: